

Практическая работа: «Live-центр мониторинга» (SignalR)

Общая идея

С нуля создать веб-панель, которая в реальном времени отслеживает активность пользователей: подключения, отключения, сообщения, статус онлайн. Всё работает без перезагрузки страницы и без периодических опросов сервера — только SignalR.

Стек: ASP.NET Core, SignalR, HTML + чистый JavaScript.

Подготовка (до заданий)

Создать проект, настроить SignalR, CORS, статические файлы и маппинг хаба.

Требования к подготовке:

- Создать пустой ASP.NET Core проект.
 - Подключить SignalR в сервисах и замапить хаб на URL.
 - Разрешить CORS для фронтенда.
 - Создать HTML-страницу с подключением библиотеки SignalR (через CDN).
 - Создать пустой класс хаба, унаследованный от Hub.
-

Задание 1. Подключение и широковещание (Базовый хаб)

Задача: реализовать двусторонний обмен сообщениями между клиентами через хаб.

Что нужно реализовать на сервере:

- Переопределить метод, который срабатывает при подключении нового клиента. В нём отправить всем остальным клиентам событие о том, что новый пользователь подключился, передав его уникальный идентификатор подключения.
- Переопределить метод, который срабатывает при отключении клиента. В нём отправить всем событие о том, что пользователь отключился.
- Создать метод, который принимает текст сообщения от клиента и рассылает его всем подключённым клиентам вместе с идентификатором отправителя.

Что нужно реализовать на клиенте:

- Создать соединение с хабом и запустить его.
- Настроить слушатели для трёх событий: подключение пользователя, отключение пользователя, получение сообщения. При каждом событии — добавлять запись в ленту событий на странице.
- Добавить поле ввода и кнопку. При нажатии на кнопку — вызвать метод хаба, передав текст из поля ввода.

Контрольная точка:

- Откройте две вкладки. Отправьте сообщение из одной — оно мгновенно появляется в обеих.
 - При открытии третьей вкладки в первых двух появляется уведомление о новом подключении.
 - При закрытии вкладки — уведомление об отключении.
-

Задание 2. Группы: комнаты для адресной рассылки

Задача: реализовать возможность создавать комнаты и отправлять сообщения только участникам конкретной комнаты.

Что нужно реализовать на сервере:

- Метод для вступления в комнату: добавляет текущее подключение в группу с указанным именем. Подтверждает вызвавшему клиенту, что он успешно вошёл.
- Метод для выхода из комнаты: удаляет подключение из группы. Подтверждает вызвавшему, что он вышел.
- Метод для отправки сообщения в комнату: принимает имя комнаты и текст, отправляет событие только участникам этой группы, передавая идентификатор отправителя и текст.

Что нужно реализовать на клиенте:

- Поле для ввода названия комнаты и две кнопки: «Войти» и «Выйти».
- Отдельное поле и кнопка для отправки сообщения в текущую комнату.
- Слушатели для событий: подтверждение вступления в комнату, подтверждение выхода, получение сообщения из комнаты. Сообщения из комнаты должны визуально отличаться от широковещательных (другой цвет или пометка).

Контрольная точка:

- Откройте три вкладки. Две входят в комнату «alpha», третья — в «beta».
 - Сообщения в «alpha» получают только первые две вкладки. Третью его не видит.
 - Сообщение в «beta» получает только третья.
-

Задание 3. Отслеживание состояния: счётчик онлайн и хранилище подключений

Задача: сервер должен знать, сколько клиентов подключено, и сообщать всем актуальное число.

Что нужно реализовать на сервере:

- Создать отдельный сервис (зарегистрированный как singleton), который через внедрение зависимостей получает контекст хаба. Этот сервис хранит словарь всех активных подключений: ключ — идентификатор подключения, значение — имя комнаты (пусто, если клиент ни в какой комнате).
- В сервисе реализовать три метода: регистрация нового подключения, удаление подключения, обновление комнаты для подключения.
- При регистрации и удалении — отправлять всем клиентам событие с актуальным количеством онлайн.
- Реализовать метод для отправки системного сообщения всем клиентам через контекст хаба.
- В хабе внедрить сервис и вызывать его методы в обработчиках подключения и отключения. При вступлении в комнату — обновлять запись в сервисе.

Что нужно реализовать на клиенте:

- Элемент на странице, показывающий количество онлайн.
- Слушатель события обновления счётчика — обновляет число на странице.
- Слушатель системных сообщений — добавляет их в ленту с пометкой «Система».
- Кнопку, которая делает обычный HTTP POST-запрос к серверу. На сервере этот запрос вызывает метод сервиса для отправки системного сообщения всем клиентам.

Контрольная точка:

- Откройте три вкладки — счётчик показывает 3.
- Закройте одну — счётчик уменьшается до 2 (с небольшой задержкой — это нормально, SignalR не сразу замечает отключение).

- Нажмите кнопку на странице — все клиенты получают системное сообщение, доставленное через SignalR, но вызванное обычным HTTP-запросом.
-

Задание 4. Переподключение и восстановление состояния

Задача: при временном разрыве связи клиент должен автоматически переподключиться и снова вступить в комнату, в которой был до разрыва.

Что нужно реализовать на клиенте:

- Настроить автоматическое переподключение с задержками: немедленно, затем через 2, 5, 10 и 30 секунд.
- Переменную, хранящую имя текущей комнаты.
- При вступлении в комнату — сохранять имя в эту переменную. При выходе — очищать.
- Обработчик события «переподключение начато»: показать в статусе на странице «Переподключение...».
- Обработчик события «переподключение выполнено»: показать «Подключено» и, если в переменной есть имя комнаты, заново вызвать метод хаба для вступления в эту комнату.
- Обработчик события «соединение закрыто окончательно»: показать «Соединение потеряно».

Контрольная точка:

- Откройте вкладку, войдите в комнату «alpha».
 - Имитируйте разрыв: в DevTools браузера переведите сеть в Offline.
 - Через несколько секунд статус меняется на «Переподключение...».
 - Верните сеть в Online. Статус — «Подключено». Клиент снова получает сообщения из комнаты «alpha».
 - Проверьте: если не сохранять имя комнаты и не вызывать повторное вступление — после переподключения сообщения из комнаты не доходят.
-

Задание 5. Адресная отправка: приватные сообщения и внешние уведомления

Задача: реализовать отправку сообщений конкретному клиенту по его идентификатору и отправку уведомлений в конкретную комнату из внешнего HTTP-запроса.

Что нужно реализовать на сервере:

- В хабе: метод, который принимает идентификатор целевого клиента и текст. Отправляет сообщение только этому клиенту. Отправителю возвращает подтверждение.
- В сервисе: метод для отправки сообщения в конкретную комнату через контекст хаба и метод для отправки приватного сообщения конкретному клиенту через контекст хаба.
- В Program.cs: два HTTP-эндпоинта (POST). Первый принимает имя комнаты и текст, вызывает метод сервиса для отправки в комнату. Второй принимает идентификатор клиента и текст, вызывает метод сервиса для приватной отправки.

Что нужно реализовать на клиенте:

- Отображение собственного идентификатора подключения на странице (чтобы можно было скопировать).
- Поле для ввода идентификатора получателя и поле текста. Кнопка «Отправить приватно» — вызывает метод хаба.
- Кнопку «Внешнее уведомление в комнату» — делает HTTP POST к эндпоинту, передаёт имя комнаты и текст.
- Слушатель приватных сообщений — отображает их с визуальным отличием (другой цвет, пометка «Приватное»).

Контрольная точка:

- Две вкладки. Скопируйте идентификатор из первой. Вторая отправляет приватное сообщение первой — получает только первая.
 - Нажмите «Внешнее уведомление в комнату» — все участники указанной комнаты получают сообщение, хотя оно было инициировано обычным HTTP-запросом, а не вызовом хаба.
-

Дополнительные задания (по уровням) на оценку «отлично»

Доп. задание — Лёгкий уровень: «Печатает...»

Сценарий: когда пользователь начинает вводить сообщение в поле, остальные участники его комнаты видят индикатор «Печатает...».

Требования:

- В хабе: метод, который принимает имя комнаты и отправляет событие «печатает» всем участникам комнаты, кроме отправителя.
- На клиенте: при вводе в поле (с задержкой около 500мс, чтобы не спамить сервер) вызывать метод хаба.
- На клиенте: слушатель события «печатает» — показывает текст «Пользователь печатает...» рядом с полем ввода. Через 2 секунды без новых событий — индикатор исчезает.

Контрольная точка: две вкладки в одной комнате. Одна начинает печатать — вторая видит индикатор. Перестала печатать — через 2 секунды индикатор пропадает.

Доп. задание — Средний уровень: «Глобальный статус пользователей»

Сценарий: панель показывает не просто число онлайн, а список всех подключённых пользователей с их текущей комнатой.

Требования:

- Сервис хранит не просто число, а словарь: идентификатор подключения → имя комнаты (или «без комнаты»).
- При каждом подключении, отключении, вступлении в комнату или выходе из неё — сервис отправляет всем клиентам обновлённый полный список: массив объектов с идентификатором и комнатой.
- На клиенте: список рендерится в виде таблицы или списка. При каждом обновлении — перерисовывается.
- При нажатии на пользователя из списка — его идентификатор подставляется в поле «Кому» для приватного сообщения.

Контрольная точка: открыли три вкладки. Видите всех трёх в списке. Один вошёл в комнату «alpha» — в списке у него появилась комната. Один закрыл вкладку — исчез из списка. Нажали на пользователя — его идентификатор вставился в поле получателя.

Доп. задание — Сложный уровень: «Потоковая передача данных (Streaming)»

Сценарий: сервер генерирует поток случайных чисел и непрерывно отправляет их клиенту. Клиент строит из них живой график или выводит в список. Поток можно остановить.

Требования:

- В хабе: метод, возвращающий поток данных (асинхронный перечислимый тип). Метод принимает параметр — максимальное значение числа. Каждые 500 миллисекунд генерирует случайное число и возвращает его. Цикл останавливается при отмене (через токен отмены).
- На клиенте: кнопка «Старт» — открывает поток и подписывается на получение данных, ошибку и завершение. Каждое полученное число добавляется в список или отображается на простом графике.
- На клиенте: кнопка «Стоп» — отменяет подписку на поток. После остановки числа перестают приходить.
- При закрытии вкладки — поток на сервере должен остановиться автоматически (токен отмены срабатывает при отключении клиента).

Контрольная точка: нажали «Старт» — числа начинают приходить каждые 500мс. Нажали «Стоп» — поток остановлен, новые числа не приходят. Закрыли вкладку — на сервере цикл завершился, ошибок нет.
